

SERVICIO NACIONAL DE APRENDIZAJE – SENA
Centro de Gestión de Mercados, Logística y Tecnologías de la Información
Tecnólogo en Análisis y Desarrollo de Software

**INFORME TÉCNICO DE PLAN DE TRABAJO
HERRAMIENTAS Y ESTÁNDARES DE CODIFICACIÓN
PARA LA CONSTRUCCIÓN DE SOFTWARE**

Proyecto: Trueque Match
Aplicación móvil y web de intercambio sin dinero

Evidencia: GA7-220501096-AA1-EV02

Estudiante: Gerson Jonnathan López Oviedo
Ficha: 3186647

Instructor: Arnaldo Alfonso Montiel Brun

Bogotá D.C., Mayo de 2026

TABLA DE CONTENIDO

TABLA DE CONTENIDO	2
1. Introducción	4
2. Objetivo.....	5
2.1 Objetivo General	5
2.2 Objetivos Específicos.....	5
3. Estándares de Codificación	6
3.1 ¿Qué son los Estándares de Codificación?	6
3.2 ¿Por qué son Necesarios los Estándares de Codificación?	6
3.2.1 Legibilidad y Comprensión del Código.....	6
3.2.2 Reducción de Errores.....	7
3.2.3 Facilita la Revisión de Código (Code Review)	7
3.2.4 Escalabilidad del Proyecto	7
3.2.5 Automatización y Calidad Continua	7
3.3 Prácticas Recomendadas para el Uso de Estándares de Codificación	8
3.3.1 Definir los Estándares al Inicio del Proyecto	8
3.3.2 Usar Herramientas de Análisis Estático (Linters)	8
3.3.3 Configurar Formateadores Automáticos de Código	9
3.3.4 Integrar los Estándares en el Control de Versiones	9
3.3.5 Documentar el Código con JSDoc.....	9
3.3.6 Convenciones de Nomenclatura	10
4. Herramientas de Versionamiento	11
4.1 ¿Qué es el Versionamiento de Software?.....	11
4.2 Tipos de Sistemas de Control de Versiones	11
4.2.1 Sistemas Locales	11
4.2.2 Sistemas Centralizados (CVCS).....	11
4.2.3 Sistemas Distribuidos (DVCS).....	11
4.3 Herramientas Principales de Versionamiento	12
4.3.1 Git.....	12
4.3.2 GitHub.....	13
4.3.3 GitLab	13
4.3.4 Bitbucket.....	13
4.3.5 Subversion (SVN)	13
4.4 Comparativa de Herramientas de Versionamiento	14

5. Selección de Herramientas para Trueque Match	15
5.1 Contexto del Proyecto	15
5.2 Herramientas Seleccionadas	15
5.2.1 Git como Sistema de Control de Versiones	15
5.2.2 GitHub como Plataforma de Repositorio Remoto	16
5.2.3 ESLint como Herramienta de Análisis Estático.....	16
5.2.4 Prettier como Formateador de Código	17
5.2.5 Conventional Commits como Estándar de Mensajes	17
5.3 Justificación Final de la Selección	18
6. Conclusiones	19
7. Referencias	20

1. Introducción

El desarrollo de software moderno exige no solo habilidades técnicas en programación, sino también la adopción de metodologías y herramientas que garanticen la calidad, trazabilidad y mantenibilidad del código fuente a lo largo del ciclo de vida de un proyecto. En este contexto, los estándares de codificación y las herramientas de versionamiento constituyen pilares fundamentales en cualquier proceso de construcción de software profesional.

El presente informe técnico corresponde a la evidencia GA7-220501096-AA1-EV01 del programa Tecnólogo en Análisis y Desarrollo de Software del SENA, y tiene como propósito investigar, analizar y seleccionar las herramientas y estándares de codificación más adecuados para el desarrollo del proyecto Trueque Match, una aplicación móvil y web orientada a facilitar el intercambio de bienes, servicios, conocimientos y experiencias entre personas sin necesidad de dinero.

A través de este documento se abordarán los conceptos fundamentales de los estándares de codificación, su importancia en proyectos de software reales, las prácticas recomendadas por la industria, las herramientas disponibles con ejemplos concretos, y finalmente la selección justificada de las herramientas que mejor se adaptan a las necesidades técnicas y contextuales del proyecto Trueque Match.

2. Objetivo

2.1 Objetivo General

Elaborar un informe técnico que identifique, analice y seleccione las herramientas y estándares de codificación más apropiados para la construcción del software Trueque Match, garantizando buenas prácticas de desarrollo, trazabilidad del código y trabajo colaborativo efectivo.

2.2 Objetivos Específicos

- Definir qué son los estándares de codificación y su rol dentro del desarrollo de software.
- Explicar por qué los estándares de codificación son necesarios en proyectos profesionales.
- Describir las prácticas recomendadas para el uso de estándares de codificación.
- Identificar y comparar las principales herramientas de versionamiento disponibles en la industria.
- Seleccionar y justificar las herramientas de versionamiento más adecuadas para el proyecto Trueque Match.

3. Estándares de Codificación

3.1 ¿Qué son los Estándares de Codificación?

Los estándares de codificación son un conjunto de reglas, convenciones y directrices acordadas que definen la forma en que el código fuente debe ser escrito dentro de un proyecto o equipo de desarrollo. Según Pressman y Maxim (2021), estos estándares abarcan aspectos como la nomenclatura de variables, la indentación, la longitud máxima de las líneas de código, la estructura de los comentarios, la organización de los archivos y la forma en que se documentan las funciones o módulos.

De manera análoga, si un equipo de construcción civil trabaja con planos y normas técnicas unificadas, un equipo de desarrollo de software trabaja con estándares de codificación para asegurar que todos hablen el mismo idioma al escribir código. Sin estos acuerdos, el proyecto se convierte en una torre de Babel donde nadie entiende el trabajo de los demás.

En el ámbito del desarrollo web y móvil con JavaScript y TypeScript, los estándares más reconocidos incluyen la guía de estilo de Airbnb JavaScript Style Guide, las convenciones de Google JavaScript Style Guide y las recomendaciones del consorcio W3C para HTML y CSS. Para proyectos con React y React Native, se siguen además las convenciones propias del ecosistema de Meta (antes Facebook), tales como el uso de componentes funcionales, hooks, y la separación clara entre lógica de negocio y presentación.

3.2 ¿Por qué son Necesarios los Estándares de Codificación?

La necesidad de contar con estándares de codificación responde a múltiples factores críticos en el desarrollo de software. Sommerville (2016) argumenta que la mantenibilidad del software depende directamente de la legibilidad y coherencia del código, características que solo se logran cuando todos los integrantes del equipo siguen las mismas convenciones.

A continuación se presentan las razones principales por las que los estándares son indispensables:

3.2.1 Legibilidad y Comprensión del Código

Cuando múltiples desarrolladores trabajan en el mismo proyecto, cada uno tiene sus propios hábitos de codificación. Sin estándares, el código de una persona puede resultar incomprensible para otra. Un estándar garantiza que cualquier miembro del equipo pueda leer, comprender y modificar cualquier parte del código sin necesidad de consultar permanentemente al autor original.

3.2.2 Reducción de Errores

Muchas convenciones de codificación nacieron precisamente para prevenir errores comunes. Por ejemplo, la regla de siempre usar llaves en los bloques if/else en JavaScript previene bugs sutiles que ocurren cuando se añade una línea sin envolver correctamente. El linter ESLint, que aplica estas reglas automáticamente, puede detectar estos errores antes de que el código llegue a producción.

3.2.3 Facilita la Revisión de Código (Code Review)

En equipos de desarrollo, es común que un desarrollador revise el código de otro antes de fusionarlo al repositorio principal. Esta práctica, conocida como code review o revisión de pares, se vuelve considerablemente más eficiente cuando el código sigue convenciones predefinidas, ya que el revisor puede concentrarse en la lógica y el funcionamiento, no en discutir si las variables deben ir en camelCase o snake_case.

3.2.4 Escalabilidad del Proyecto

A medida que un proyecto crece, la cantidad de código aumenta exponencialmente. Un proyecto que comienza con 500 líneas de código puede llegar a tener 50.000 o más en producción. Sin estándares definidos desde el inicio, mantener y escalar ese código se convierte en una tarea extraordinariamente costosa en tiempo y recursos. Los estándares permiten que nuevos desarrolladores se integren al proyecto rápidamente y contribuyan de manera productiva.

3.2.5 Automatización y Calidad Continua

Los estándares de codificación permiten la implementación de herramientas de análisis estático de código, como ESLint o SonarQube, que verifican automáticamente el cumplimiento de las reglas cada vez que se realiza un cambio en el código. Esto crea una red de seguridad automatizada que reduce la deuda técnica y asegura niveles mínimos de calidad de manera continua.

La siguiente tabla resume las principales necesidades que atienden los estándares de codificación:

Necesidad	Problema que Resuelve	Beneficio Obtenido
Legibilidad	Código incomprensible entre miembros	Reducción del tiempo de onboarding
Consistencia	Estilos distintos en el mismo proyecto	Código uniforme y profesional
Detección de errores	Bugs por malas prácticas	Menor deuda técnica
Mantenibilidad	Dificultad para modificar código antiguo	Cambios más rápidos y seguros
Colaboración	Conflictos de estilo en revisiones	Code reviews más eficientes
Automatización	Revisión manual ineficiente	CI/CD con control de calidad automático

3.3 Prácticas Recomendadas para el Uso de Estándares de Codificación

La adopción de estándares de codificación no consiste únicamente en elegir un conjunto de reglas y publicarlas en un documento. Para que sean efectivos, deben implementarse de manera sistemática a lo largo de todo el ciclo de vida del proyecto. Las siguientes son las prácticas más recomendadas por la industria:

3.3.1 Definir los Estándares al Inicio del Proyecto

Uno de los errores más comunes es intentar implementar estándares cuando el proyecto ya lleva semanas o meses de desarrollo. Refactorizar código existente para que cumpla con las nuevas reglas es costoso y riesgoso. La práctica recomendada es definir los estándares antes de escribir la primera línea de código funcional.

3.3.2 Usar Herramientas de Análisis Estático (Linters)

Un linter es una herramienta que analiza el código fuente en busca de errores, violaciones de estilo y patrones problemáticos sin necesidad de ejecutar el programa. Para JavaScript y TypeScript, ESLint es el estándar de facto. Se configura mediante un archivo

.eslintrc.json o .eslintrc.js en la raíz del proyecto y puede integrarse directamente con el editor de código Visual Studio Code para mostrar advertencias en tiempo real mientras se escribe.

Ejemplo de configuración básica de ESLint para un proyecto React Native:

```
// Archivo: .eslintrc.json
{
  "extends": ["eslint:recommended", "plugin:react/recommended"],
  "rules": {
    "no-unused-vars": "warn",
    "eqeqeq": "error",
    "semi": ["error", "always"],
    "quotes": ["error", "single"]
  }
}
```

3.3.3 Configurar Formateadores Automáticos de Código

Prettier es un formateador de código que aplica automáticamente un estilo consistente al guardar un archivo. A diferencia de ESLint, que detecta problemas de lógica y estilo, Prettier se enfoca exclusivamente en el formato visual del código: indentación, longitud de líneas, uso de comillas, espaciado, entre otros. La combinación de ESLint y Prettier es una práctica estándar en proyectos modernos de JavaScript.

3.3.4 Integrar los Estándares en el Control de Versiones

Una práctica avanzada pero muy recomendada es la integración de los estándares con el sistema de control de versiones. Herramientas como Husky permiten ejecutar el linter automáticamente antes de cada commit, de tal manera que si el código no cumple con los estándares, el commit es rechazado. Esto garantiza que el repositorio siempre contenga código que respeta las convenciones definidas.

3.3.5 Documentar el Código con JSDoc

Los comentarios de documentación siguen un formato estándar que permite generar documentación técnica automáticamente. En JavaScript, el estándar JSDoc define cómo deben estructurarse los comentarios de funciones, parámetros y valores de retorno. Esta práctica es especialmente importante en proyectos que serán mantenidos por múltiples personas o que tienen ciclos de vida largos.

```

/**
 * Autentica al usuario con correo y contraseña
 * @param {string} correo - Dirección de correo electrónico
 * @param {string} password - Contraseña del usuario
 * @returns {Promise<string>} Token JWT de la sesión
 */

```

3.3.6 Convenciones de Nomenclatura

Una de las áreas más importantes de los estándares de codificación es la nomenclatura. Las convenciones más utilizadas en JavaScript y TypeScript son:

Tipo de Identificador	Convención	Ejemplo
Variables y funciones	camelCase	nombreUsuario, calcularTotal()
Componentes React	PascalCase	PantallaLogin, TarjetaOferta
Constantes globales	UPPER_SNAKE_CASE	MAX_INTENTOS, URL_BASE
Archivos de componentes	PascalCase.tsx	PantallaHome.tsx
Archivos de utilidades	camelCase.ts	validarEmail.ts
Interfaces TypeScript	PascalCase con I-	IUsuario, IOferta
Tipos TypeScript	PascalCase	TipoCategoria, EstadoTrueque

4. Herramientas de Versionamiento

4.1 ¿Qué es el Versionamiento de Software?

El versionamiento o control de versiones es la práctica de registrar y gestionar los cambios realizados al código fuente de un software a lo largo del tiempo. Un sistema de control de versiones (VCS, por sus siglas en inglés) permite a los desarrolladores guardar el historial completo de modificaciones, revertir cambios, trabajar en paralelo en diferentes funcionalidades y fusionar el trabajo de múltiples colaboradores sin perder información.

Chacon y Straub (2014) definen el control de versiones como un sistema que registra los cambios realizados sobre un archivo o conjunto de archivos a lo largo del tiempo, de tal manera que sea posible recuperar versiones específicas en cualquier momento. Esta capacidad es especialmente crítica en proyectos de software donde un error introducido en una nueva versión puede afectar a todos los usuarios del sistema.

4.2 Tipos de Sistemas de Control de Versiones

4.2.1 Sistemas Locales

Los sistemas de control de versiones más simples funcionan localmente en el equipo del desarrollador. Un ejemplo histórico es RCS (Revision Control System), que guarda parches de las diferencias entre versiones en formato especial. Aunque funcional, este enfoque presenta el grave problema de no permitir la colaboración entre múltiples desarrolladores.

4.2.2 Sistemas Centralizados (CVCS)

Los sistemas de control de versiones centralizados, como Subversion (SVN) y CVS, resuelven el problema de la colaboración mediante un servidor central que almacena todas las versiones del proyecto. Los desarrolladores realizan checkout del código, trabajan localmente y luego hacen commit al servidor central. El mayor riesgo de este modelo es el servidor único: si falla, todo el equipo queda bloqueado y si se corrompe sin respaldo, se pierde toda la historia del proyecto.

4.2.3 Sistemas Distribuidos (DVCS)

Los sistemas distribuidos, como Git y Mercurial, superan las limitaciones de los sistemas centralizados. En un DVCS cada desarrollador tiene una copia completa del

repositorio, incluida toda la historia del proyecto. Esto significa que casi todas las operaciones son locales y extremadamente rápidas, y que el proyecto puede continuar aunque el servidor central falle temporalmente.

4.3 Herramientas Principales de Versionamiento

4.3.1 Git

Git es el sistema de control de versiones distribuido más utilizado en el mundo. Fue creado en 2005 por Linus Torvalds, el mismo creador del kernel de Linux, con el objetivo de gestionar el desarrollo colaborativo de ese sistema operativo. Según la encuesta anual de Stack Overflow 2023, el 93.87% de los desarrolladores profesionales utiliza Git como su herramienta principal de control de versiones.

Git introduce el concepto de área de preparación (staging area o índice), que permite a los desarrolladores preparar exactamente qué cambios van a incluir en el próximo commit, ofreciendo un control muy granular sobre la historia del proyecto.

Comandos fundamentales de Git:

Comando	Función
git init	Inicializa un nuevo repositorio Git en el directorio actual
git clone <url>	Clona un repositorio remoto en el equipo local
git add <archivo>	Agrega archivos al área de preparación (staging)
git commit -m 'mensaje'	Guarda los cambios en el historial con un mensaje descriptivo
git push origin main	Sube los commits locales al repositorio remoto
git pull	Descarga y fusiona los cambios del repositorio remoto
git branch <nombre>	Crea una nueva rama de trabajo
git checkout <rama>	Cambia a una rama existente
git merge <rama>	Fusiona una rama con la rama actual
git log --oneline	Muestra el historial de commits de forma resumida

4.3.2 GitHub

GitHub es la plataforma de alojamiento de repositorios Git más popular del mundo, con más de 100 millones de desarrolladores y 420 millones de repositorios según datos de la propia plataforma (GitHub, 2024). GitHub no es simplemente un servidor para guardar código: es una plataforma completa de colaboración que incluye control de versiones, revisión de código mediante pull requests, gestión de issues, automatización con GitHub Actions, wikis de documentación y tableros de proyectos.

Una de las características más poderosas de GitHub para equipos de desarrollo es el flujo de trabajo basado en ramas (branching strategy). El modelo más utilizado es GitHub Flow, que propone mantener siempre una rama principal estable (main o master) y crear ramas separadas para cada nueva funcionalidad o corrección de error, las cuales se fusionan mediante pull requests después de ser revisadas por otros miembros del equipo.

4.3.3 GitLab

GitLab es una plataforma DevOps completa de código abierto que ofrece funcionalidades similares a GitHub, pero con la ventaja de poder instalarse en servidores propios (self-hosted). GitLab se distingue por integrar en una sola plataforma todas las etapas del ciclo DevOps: desde la planificación y el control de versiones, hasta la integración continua, el despliegue continuo y el monitoreo. Es ampliamente utilizado en empresas que requieren mayor control sobre sus datos y su infraestructura.

4.3.4 Bitbucket

Bitbucket es la plataforma de alojamiento de repositorios Git de Atlassian, la empresa detrás de Jira y Confluence. Su principal ventaja competitiva radica en su integración nativa con las herramientas del ecosistema Atlassian, lo que la hace especialmente atractiva para equipos que ya usan Jira para la gestión de proyectos y Confluence para la documentación técnica. Bitbucket ofrece repositorios privados ilimitados en su plan gratuito.

4.3.5 Subversion (SVN)

Apache Subversion, comúnmente conocido como SVN, es un sistema de control de versiones centralizado que fue el estándar de la industria antes de la masificación de Git. A diferencia de Git, en SVN el historial completo del proyecto reside en el servidor central y los

desarrolladores trabajan con copias de trabajo locales. Aunque ha perdido terreno frente a Git, SVN sigue siendo utilizado en algunas organizaciones con proyectos legacy o con requisitos estrictos de control centralizado.

4.4 Comparativa de Herramientas de Versionamiento

La siguiente tabla presenta una comparativa objetiva de las principales herramientas de versionamiento analizadas, considerando los criterios más relevantes para un proyecto de desarrollo de software:

Criterio	Git	GitHub	GitLab	Bitbucket	SVN
Tipo	DVCS local	Plataforma Git	Plataforma DevOps	Plataforma Git	CVCS
Modelo	Distribuido	Distribuido	Distribuido	Distribuido	Centralizado
Costo base	Gratuito	Gratuito/Pro	Gratuito/Premium	Gratuito/Premium	Gratuito
CI/CD integrado	No nativo	GitHub Actions	GitLab CI/CD	Pipelines	No nativo
Trabajo offline	Sí	Sí	Sí	Sí	Limitado
Pull Requests	No aplica	Sí	Merge Requests	Sí	No nativo
Gestión de issues	No	Sí	Sí	Sí	No
Self-hosted	Sí (local)	GitHub Enterprise	Sí (gratuito)	Datacenter	Sí
Curva aprendizaje	Media-Alta	Baja-Media	Media	Baja-Media	Baja
Popularidad 2024	93.87%	#1 plataforma	#2 plataforma	#3 plataforma	< 5%

5. Selección de Herramientas para Trueque Match

5.1 Contexto del Proyecto

Trueque Match es una aplicación móvil y web desarrollada con React Native + Expo para el cliente móvil Android, React.js para el cliente web, y Node.js + Express para el backend. El proyecto es desarrollado por un estudiante de manera individual en el marco del programa Tecnólogo en Análisis y Desarrollo de Software del SENA, utilizando un computador portátil ACER Core 7 con Windows y un dispositivo de prueba Redmi 9 para la aplicación móvil.

Dado este contexto, la selección de herramientas debe considerar los siguientes criterios de prioridad:

- Gratuidad: el proyecto no cuenta con presupuesto para licencias de software.
- Facilidad de uso: el desarrollador está en proceso de formación y necesita herramientas con curva de aprendizaje razonable.
- Compatibilidad: las herramientas deben funcionar correctamente en el ecosistema Windows + JavaScript/TypeScript + React Native.
- Integración: las herramientas deben poder conectarse entre sí para crear un flujo de trabajo cohesionado.
- Adopción en la industria: priorizar herramientas ampliamente utilizadas en el mercado laboral colombiano e internacional.

5.2 Herramientas Seleccionadas

5.2.1 Git como Sistema de Control de Versiones

Git es la selección natural e indiscutible como sistema de control de versiones para Trueque Match. Su naturaleza distribuida permite trabajar con total funcionalidad incluso sin conexión a internet, lo cual es relevante dado que el estudiante trabaja desde casa con acceso a internet que puede ser intermitente. Además, Git es la base tecnológica sobre la que se construyen las demás plataformas consideradas.

La adopción de Git en Trueque Match permitirá implementar una estrategia de ramas clara: una rama main con el código estable, una rama develop para integración de nuevas funcionalidades, y ramas feature/* para el desarrollo de cada pantalla o módulo específico. Por ejemplo: feature/pantalla-login, feature/pantalla-home, feature/modelo-bd.

```
# Flujo de trabajo Git para Trueque Match
git checkout -b feature/pantalla-home # nueva función
git add . # preparar cambios
git commit -m "feat: agrega pantalla Home con categorías"
git push origin feature/pantalla-home # subir al remoto
git checkout develop && git merge feature/pantalla-home
```

5.2.2 GitHub como Plataforma de Repositorio Remoto

GitHub es la plataforma seleccionada para alojar el repositorio remoto de Trueque Match. Las razones que justifican esta elección por encima de alternativas como GitLab o Bitbucket son:

- La cuenta del estudiante (gjlo) ya está configurada en la plataforma Expo EAS, lo cual facilita la integración con el proceso de compilación del APK.
- GitHub ofrece repositorios privados ilimitados en su plan gratuito, lo que permite proteger el código del proyecto durante su desarrollo.
- GitHub Actions, el sistema de integración continua de la plataforma, permite automatizar en el futuro la ejecución del linter y las pruebas cada vez que se realice un push al repositorio.
- GitHub es la plataforma más demandada en el mercado laboral, por lo que su dominio constituye una competencia profesional directamente aplicable.
- El ecosistema de GitHub cuenta con integraciones nativas con las principales herramientas del stack tecnológico de Trueque Match, incluyendo Expo, VS Code y Node.js.

5.2.3 ESLint como Herramienta de Análisis Estático

Para garantizar el cumplimiento de los estándares de codificación en el código JavaScript y TypeScript del proyecto, se selecciona ESLint como herramienta de análisis estático. ESLint se configura con el conjunto de reglas recomendado para React Native y

TypeScript, y se integra con la extensión ESLint de Visual Studio Code para proporcionar retroalimentación visual inmediata mientras se escribe el código.

5.2.4 Prettier como Formateador de Código

Prettier complementa a ESLint encargándose del formato visual del código. Su configuración para Trueque Match incluirá: uso de comillas simples, punto y coma al final de las instrucciones, indentación de 2 espacios, y longitud máxima de línea de 100 caracteres. Prettier se configura para ejecutarse automáticamente al guardar un archivo en VS Code, eliminando la necesidad de formatear el código manualmente.

5.2.5 Conventional Commits como Estándar de Mensajes

Para mantener un historial de cambios legible y estructurado, Trueque Match adoptará la especificación Conventional Commits para los mensajes de los commits. Este estándar define un formato claro: tipo(alcance): descripción breve. Los tipos más utilizados incluyen feat para nuevas funcionalidades, fix para correcciones de errores, docs para cambios en documentación, style para cambios de formato, y refactor para reestructuración del código sin cambios funcionales.

Tipo de Commit	Uso	Ejemplo Real en Trueque Match
feat	Nueva funcionalidad	feat(auth): implementa login con JWT
fix	Corrección de error	fix(chat): corrige badge de mensajes no leídos
docs	Documentación	docs: agrega JSDoc a funciones de API
style	Formato de código	style: aplica prettier a pantalla Home
refactor	Reestructuración	refactor(db): normaliza tabla TRUEQUE a 3FN
test	Pruebas	test: agrega test unitario para validarEmail()
chore	Tareas de mantenimiento	chore: actualiza dependencias de Expo SDK

5.3 Justificación Final de la Selección

La combinación Git + GitHub + ESLint + Prettier + Conventional Commits representa el stack de herramientas de versionamiento y estándares de codificación más apropiado para Trueque Match por las siguientes razones finales:

En primer lugar, esta combinación de herramientas es completamente gratuita en su plan básico, lo que la hace accesible para un proyecto académico sin presupuesto. En segundo lugar, es el stack más utilizado en la industria del desarrollo de software con JavaScript y React Native en Colombia y a nivel internacional, lo que significa que las competencias adquiridas son directamente aplicables en el mercado laboral. En tercer lugar, todas las herramientas se integran de manera nativa entre sí: ESLint y Prettier se integran con VS Code, VS Code tiene integración nativa con Git y GitHub, y Expo EAS se conecta directamente con repositorios de GitHub para ejecutar las compilaciones del APK.

Finalmente, este conjunto de herramientas garantiza que el código de Trueque Match sea legible, consistente y mantenible a lo largo de todo el proceso de desarrollo, lo que resulta en un producto de mayor calidad y en un portafolio más sólido para el estudiante.

Herramienta	Rol en el Proyecto	Justificación
Git	VCS distribuido local	Gratuito, estándar industria, trabajo offline
GitHub	Repositorio remoto y CI/CD	Integración con Expo, plan gratuito, portafolio
ESLint	Análisis estático de código	Detecta errores en JS/TS automáticamente
Prettier	Formateador automático	Garantiza consistencia visual del código
Conventional Commits	Estándar de mensajes Git	Historial legible y semántico

6. Conclusiones

Los estándares de codificación y las herramientas de versionamiento no son elementos opcionales en el desarrollo de software profesional, sino componentes fundamentales que determinan la calidad, mantenibilidad y escalabilidad de cualquier proyecto. A través de este informe técnico se ha evidenciado que su adopción desde las etapas iniciales del desarrollo impacta directamente en la eficiencia del trabajo y en la reducción de la deuda técnica.

Para el proyecto Trueque Match, la selección de Git como sistema de control de versiones distribuido, GitHub como plataforma de alojamiento y colaboración, ESLint y Prettier como herramientas de calidad de código, y la especificación Conventional Commits como estándar de mensajes, representa la decisión técnica más fundamentada considerando el contexto académico, las tecnologías utilizadas y los requerimientos del proyecto.

La implementación de estas herramientas no solo beneficia el desarrollo actual de Trueque Match, sino que también dota al estudiante de competencias directamente aplicables en el entorno profesional del desarrollo de software en Colombia y a nivel internacional, donde estas herramientas son consideradas conocimiento fundamental en cualquier equipo de desarrollo moderno.

7. Referencias

Las siguientes referencias fueron consultadas para la elaboración de este informe técnico, siguiendo el formato APA séptima edición:

Chacon, S., y Straub, B. (2014). Pro Git (2.^a ed.). Apress. <https://git-scm.com/book/en/v2>

Eslint. (2024). ESLint - Pluggable JavaScript linter. ESLint. <https://eslint.org>

GitHub. (2024). About GitHub. GitHub Inc. <https://github.com/about>

GitLab. (2024). The DevSecOps Platform. GitLab Inc. <https://about.gitlab.com>

Meta Open Source. (2024). React Native - Learn once, write anywhere. Meta Platforms. <https://reactnative.dev>

Prettier. (2024). Prettier - Opinionated Code Formatter. Prettier. <https://prettier.io>

Pressman, R. S., y Maxim, B. R. (2021). Ingeniería del Software: Un enfoque práctico (8.^a ed.). McGraw-Hill Education.

Sommerville, I. (2016). Ingeniería de Software (10.^a ed.). Pearson Educación.

Stack Overflow. (2023). Stack Overflow Developer Survey 2023. Stack Overflow. <https://survey.stackoverflow.co/2023>

W3C. (2024). Web Content Accessibility Guidelines (WCAG) 2.1. World Wide Web Consortium. <https://www.w3.org/TR/WCAG21/>

Zhu, H. (2017). Software Design Methodology: From Principles to Architectural Styles (2.^a ed.). Elsevier Science.